

Week 3 — Lab Handout (Student)

Building Your Routing Skeleton

Time: ~90 min in-class **Work as:** Your team (5 members) **AI use this week:** **X OFF.** This is the Fundamentals Wall — write it yourself. **What you leave with:** A routing table and working stub handlers for every CRUD operation.

Why this lab matters

This is the skeleton every later feature hangs on. You are not writing database logic today — you are making sure your app can correctly *receive* every request it will ever need. Build this by hand: the Week 5 checkpoint tests whether you personally can create and fix a route without help.

Task 1 — Build your routing table (20 min)

Take your Week 2 CRUD backlog. For **each** record type, write one route per operation using RESTful conventions:

Method	Path	Handler	Story it serves
GET	/orders	listOrders	View all orders
GET	/orders/:id	showOrder	View one order
POST	/orders	createOrder	Create an order
PUT	/orders/:id	updateOrder	Edit an order
DELETE	/orders/:id	deleteOrder	Delete an order

- ☐ Every CRUD story from Deliverable 1 has a matching route.
- ☐ Methods are correct (no creating or deleting with GET).
- ☐ Save the table to `/docs/routes.md` in your repo.

Task 2 — Implement stub handlers (30 min)

For each route, write a handler that returns a **standardized placeholder response** — no database yet. Pseudocode:

```
function listOrders(request):
  return response(status = 200, body = { message: "listOrders stub" })

function createOrder(request):
  return response(status = 201, body = { message: "createOrder stub" })

function showOrder(request):
  id = request.params.id
  return response(status = 200, body = { message: "showOrder stub", id: id })
```

- ☐ Every route in your table has a working handler.
- ☐ Each returns a correct status code (200 read, 201 create, etc.).
- ☐ Route parameters (`:id`) are read and echoed back in the stub.

Agree on **one** response shape as a team now (e.g., always `{ status, data, error }`). Consistency here saves pain in Week 5.

Task 3 — Test every route (20 min)

Send a real request to each route and confirm the response.

- Use the **browser** for GET routes.
- Use **curl**, **Postman**, or **Insomnia** for POST / PUT / DELETE.
- [] Every route returns the expected status and stub body.
- ☐ A wrong method to a path (e.g., `DELETE /orders` with no id) behaves sensibly.
- ☐ Record one example request + response per route in `/docs/routes.md`.

Task 4 — Commit through the workflow (10 min)

- ☐ Each member owns at least one route on the task board.
 - ☐ Work committed on a branch and merged via a **reviewed pull request** (branch protection still applies).
 - ☐ No AI was used — note this honestly; there is no prompt log this week because there should be nothing to log.
-

End-of-Lab Checklist

- ☐ `/docs/routes.md` — full routing table + one example request/response per route
 - ☐ A working stub handler for every CRUD route, returning correct status codes
 - ☐ Route parameters read correctly
 - ☐ One consistent response shape across the team
 - ☐ Every route owned on the board; all merges via reviewed PR
-

Looking ahead to Week 4

You'll add **input validation & defensive coding** — filtering and rejecting bad input *before* it reaches your logic, and returning clear error responses. Still AI-off. Come ready to try to break your own routes.